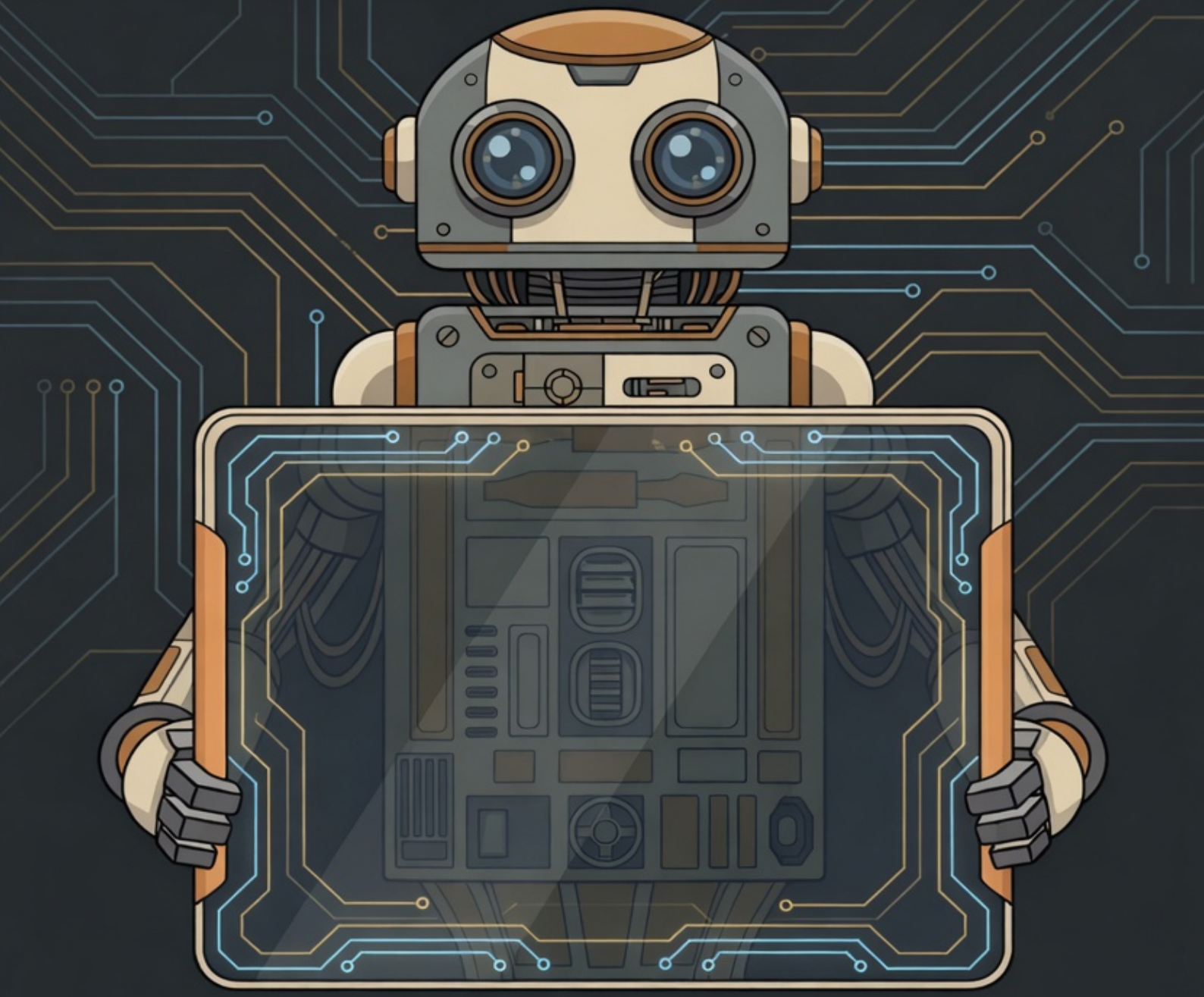
ARLIZ

Go Beyond Arrays, Memory, and Logic —
A Journey from Foundations to Frontiers

Mahdi (m-mdy-m)



ARLIZ

Arrays • Reasoning • Logic • Identity • Zero

"From ancient counting stones to quantum algorithms—
every data structure tells the story of human ingenuity."

Living First Edition

Updated: June 15, 2026

ARLIZ

ARRAYS, REASONING, LOGIC, IDENTITY, ZERO

A Living Architecture of Computing — From Voltage to Vector

Author: Mahdi (@m-mdy-m)
Location: Iran
Repository: `https://github.com/papyrxis/arliz`
Contact: bitsgenix@gmail.com

COPYRIGHT

DUAL LICENSING

PREFERRED CITATION

Mahdi. (2026). *Arliz: Arrays, Reasoning, Logic, Identity, Zero*. Retrieved from https://github.com/papyrxis/arliz

TRADEMARKS AND THIRD-PARTY NAMES

Trademarked names, logos, and product names (such as those of processor architectures, programming languages, libraries, and hardware vendors) may appear throughout this book. Rather than use a trademark symbol with every occurrence, such names are used in an editorial fashion, for identification and explanatory purposes only, and to the benefit of their respective owners, with no intention of infringement and no implication of endorsement, sponsorship, or affiliation.

WARRANTIES AND DISCLAIMERS

No Warranty: This work is provided *"AS IS"* without warranty of any kind, either expressed or implied, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. While every reasonable effort has been made to ensure the technical accuracy of the explanations, diagrams, and code examples in this book, neither the author nor any contributor can accept legal responsibility for any errors, omissions, or consequences arising from the use of this material.

Limitation of Liability: In no event shall the author or any contributor be liable for any direct, indirect, incidental, special, exemplary, or consequential damages arising from the use of this work.

Educational Purpose: This work is intended primarily for educational, reference, and research purposes.

For questions about licensing, attribution, or redistribution, please open an issue on the repository or contact bitsgenix@gmail.com.

If you are reading this in print and find it useful, consider giving the project a star on GitHub — it costs nothing and helps others find it.

Acknowledgments

To everyone who took the time to read, question, correct,
or argue with an earlier draft of this book—
thank you for making the journey sharper and more honest.

Hanieh Bakhshi – hanieh200482@gmail.com

If you contributed to Arliz—through code, review, discussion, translation, or simply by catching a mistake—and your name does not yet appear here, please open an issue or pull request.

About the Author

Mahdi (@m-mdy-m) began programming at 19 with Kotlin, then moved on to Python and JavaScript before developing a deeper interest in data structures, algorithms, and the lower-level mechanisms behind them. Over time, that interest led him to work with languages and tools including Go, C#, Haskell, Perl, C, and Fortran, with most of his work centered on tooling, research, and developer-facing infrastructure.

He is an open-source developer and backend engineer whose work focuses on modular systems, event-driven architecture, and command-line tooling. His projects include Gland, PSX, FTX, and several other open-source tools developed for practical use, experimentation, and technical exploration.

He is active in open-source communities through the Papyrxis organization on GitHub, where he develops and maintains his work in public.

Contents

Preface

Why Would Anyone Learn This Stuff?

EVERY BOOK HAS ITS ORIGIN STORY, and this one is no exception. If I were to capture the essence of creating this book in a single word, that word would be **curiosity**—though *improvised* comes as a close second. What you hold in your hands (or view on your screen) is the result of years of persistent questioning, a journey that began with a simple yet profound realization: I didn't truly understand what an array was.

This might sound trivial to some. After all, arrays are fundamental to programming, covered in every computer science curriculum, explained in countless tutorials. Yet despite encountering terms like `array`, `stack`, `queue`, `hash table`, and `tensor` repeatedly throughout my studies, I found myself increasingly frustrated by the superficial explanations typically offered. Most resources assumed you already knew what these structures fundamentally represented—their conceptual essence, their implementation mechanics, their performance characteristics, and, beneath all of that, the physical machinery that makes any of it possible.

But I wanted the *roots*. I needed to understand not just how to use an array, but what it truly meant at every level—from a voltage difference across a transistor to a tensor core multiplying matrices for a neural network. This led me to a decisive moment:

If I truly want to understand, I must build from the foundation—and follow that foundation all the way to the frontier.

And so began the journey that became Arliz.

The Name and Its Meaning

The name "Arliz" started as a somewhat arbitrary choice—I needed a title, and it sounded right. However, as the book evolved, I discovered a fitting expansion that captures its essence:

Arliz = Arrays, Reasoning, Logic, Identity, Zero

This backronym embodies the core pillars of our exploration:

- **Arrays:** The fundamental data structure we seek to understand from voltage to vector
- **Reasoning:** The systematic thinking behind data organization and algorithmic design
- **Logic:** The formal and physical principles that govern computation and data manipulation
- **Identity:** The concept of distinguishing, indexing, and assigning meaning to elements within structures
- **Zero:** The foundation from which all indexing, computation, and systematic organization originates

You may pronounce it "Ar-liz," "Array-Liz," or however feels natural to you. I personally say "ar-liz," but the pronunciation matters less than the journey it represents.

One Work, Three Volumes

Arliz was never meant to be read in a single sitting, and after enough rewrites I stopped pretending otherwise. What you are holding is not "a book"—it is one volume of a single continuous work, divided the way older technical and philosophical traditions used to divide large projects: not into arbitrary chapters of similar length, but into volumes that each stand on their own while still being unmistakably part of one larger whole. A reader of Ibn Sīnā's *al-Shifāʾ*, or of Aristotle's or al-Fārābī's treatises, did not expect to carry the entire corpus bound into a single spine; each volume was a complete stage of the argument, readable on its own terms, yet pointing forward and backward to the rest.

Arliz follows that model. The whole work walks one continuous path—from the moment a voltage appears across a transistor, to the moment that voltage becomes a bit, then a number, then an array element, and finally a structure powerful enough to run a sorting algorithm, traverse a graph, or feed a tensor into a neural network. That path is too long, and the material at each stage too dense, to fit comfortably between two covers. So the work is divided into three volumes, each one a complete stage of that path:

- **Volume I — Substrate & Signal:** How information is encoded at all, from semiconductor switching and voltage levels through bits, numbers, characters, and the raw encodings everything else is built from.

- **Volume II — Architecture & Circuitry:** The hardware that turns encoded information into computation—logic gates, memory hierarchies, processors, caches, vector units, GPUs, and the interconnects that move data between all of them.
- **Volume III — Array Odyssey:** Arrays themselves, in full: their mathematics, their memory layout, every variant and technique built on top of them, the data structures and algorithms they enable, and the parallel and large-scale systems that process them today.

This volume's Introduction lays out exactly what *this* volume covers, what it assumes you already know, and how it connects to its companions; I won't repeat that here. What matters for this preface is simply that the division into volumes is not a retreat from the project's scope—if anything, it is what makes the scope possible. Each volume can be read, revised, and improved on its own schedule, while the underlying argument remains one continuous line from voltage to vector.

What This Work Represents

Arliz is not a gentle introduction to programming, nor is it a purely theoretical treatment of data structures. It is a comprehensive technical exploration of the most fundamental data structure in computing, examined from every relevant angle, and grounded the entire way down in the physical reality of the machines that run our code.

This is a living work. As I discover better explanations, uncover new implementation details, or recognize deeper connections between concepts, each volume improves—sometimes independently of the others. Your engagement, through corrections, suggestions, and questions, makes you part of that evolution. The Introduction that follows is the practical companion to this more personal note: it covers prerequisites, difficulty, and how this volume fits into the larger work.

Mahdi
2026

> *"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."*
>
> — EDSGER W. DIJKSTRA

Introduction

ARRAYS ARE EVERYWHERE. Every image you view, every text you read, every game you play, every database query you execute, every model that recognizes your voice—arrays underlie them all. Yet despite their ubiquity, arrays remain poorly understood by most programmers. They are treated as primitive constructs, learned hastily and used mechanically, their true nature obscured by layers of abstraction stacked between a line of code and the silicon that executes it.

Arliz exists to remedy that situation. The Preface explained why I went looking for that remedy, what the name means, and why the work is divided into three volumes; this Introduction is about *this* volume specifically—what it covers, what it assumes of you, and how it connects to the rest of the work.

This Volume in the Larger Work

Arliz follows one continuous path: from how information is encoded, to the machines that compute on that information, to the array itself in all its forms. That path is divided into three volumes:

- **Volume I — Substrate & Signal** (this volume)
  We begin with the fundamental question: how is information encoded in digital systems at all? We start at the level of voltage and binary switching, then move through binary representation and Boolean algebra, number systems, integer and floating-point formats, character encoding, bitwise operations, alignment, and serialization. This volume establishes the representational substrate that determines how every array element is actually stored and manipulated at the machine level.
- **Volume II — Architecture & Circuitry**
  Picks up where this volume ends. Arrays do not exist in abstract space—they live in physical hardware with specific characteristics and constraints. Volume II examines semiconductor physics, logic gates, arithmetic circuits, memory cell design, the full memory hierarchy and cache behavior, processor pipelines,

instruction set architectures, vector and SIMD units, GPU architecture, NUMA systems, and the interconnects that tie it all together.

- **Volume III — Array Odyssey**
  The destination of the first two volumes. Arrays in full: their mathematics, memory layout, every major variant, the data structures and algorithms built on top of them, and the parallel and large-scale systems—from CPU threads to GPU kernels to distributed tensors—that process them today.

Each volume is written to be readable on its own, but the dependencies run in one direction: Volume II assumes the representational vocabulary built here in Volume I, and Volume III assumes both. If you find yourself in Volume III wondering why an operation costs what it does at the hardware level, or in Volume II wondering why a value is encoded the way it is, the earlier volume is where that question is answered.

What This Volume Covers

This volume answers a single question, asked as literally as possible: how is information encoded in digital systems, starting from nothing but a voltage difference? We begin before computing altogether—with what data *is*, and what it means for one thing to represent another—and then follow that question through binary switching, Boolean algebra, positional number systems, signed and unsigned integers, fixed- and floating-point arithmetic, character encoding and Unicode, endianness, bitwise manipulation, memory alignment, media representations (color, image, audio, video), pointers, error detection and correction, and serialization and compression. By the end of this volume, you will have every representational tool needed to understand, in Volume II, how hardware actually stores and moves these encodings, and, in Volume III, how arrays organize them into the structures that power real software.

Prerequisites

This book assumes substantial preparation:

Mathematical Maturity: You should be comfortable with discrete mathematics, linear algebra, and basic mathematical analysis. Specifically, you should have completed (or be comfortable with) the material in *Mathesis: The Mathematical Foundations of Computing*.

Algorithmic Analysis: You should understand asymptotic notation, recurrence relations, and basic complexity analysis. The material in *The Art of Algorithmic Analysis* provides the necessary foundations.

Programming Experience: You should be proficient in at least one programming language and comfortable reading code in multiple languages. We use pseudocode, C, and occasionally other languages for examples.
Without these prerequisites, you will struggle with substantial portions of this volume. With them, you are prepared for a deep, rewarding start to the journey from voltage to vector.

The Living Nature of This Work

Check the GitHub repository regularly for updates. The version you read today will be superseded by improved versions tomorrow—and because the work is now split across volumes, each one can be revised on its own schedule without forcing a reprint of the others. This is not a weakness but a strength: the work remains current, accurate, and relevant, and your corrections and questions are part of how it gets there.

A Note on Difficulty

This volume is challenging by design, even though it is, in a sense, the most "elementary" of the three—it deals with representation rather than algorithms or hardware. But representation is where every later mistake is born: a misunderstood encoding produces bugs and performance problems that no amount of algorithmic cleverness can fix later. Some sections will require sustained effort to master.

This difficulty is intentional. Genuine understanding is never cheap—it demands intellectual investment, persistent effort, and willingness to struggle with complex concepts. If you find sections difficult, that is normal. Persist. Work through examples. Understanding will come through sustained engagement, and it will pay off directly when you reach Volumes II and III.

Begin

This volume is the first stage of a longer journey—the one that starts at a voltage and, three volumes later, ends at a tensor. Everything here is foundation: necessary, occasionally dense, and built to be returned to.

Welcome to **Arliz**. Let us begin where everything begins—at the bit.

"Premature optimization is the root of all evil."

— DONALD E. KNUTH

Part I

Substrate & Signal

BEFORE WE can understand how arrays store elements, we must first understand how computers represent information itself. Every piece of data—numbers, text, images, instructions—exists as patterns of bits, and every bit begins its life as a voltage. This volume follows that single thread: from a voltage difference across a transistor, through binary encoding, to the numbers, characters, and bit-level structures that everything else in this work is built from.

> *"The choice of representation is often more important than the choice of algorithm."*
>
> — DONALD E. KNUTH

Chapter 1

The Nature of Data

What data actually is before it is encoded: the distinction between data, information, and meaning; raw measurements versus interpreted values; data as the record of a distinction or a difference; the role of an observer or interpreter in turning a physical trace into data; examples spanning natural "data" (tree rings, sediment layers, starlight) and human-made data (tally marks, ledgers, sensor readings); why data alone is inert without a scheme to read it; the seed of the idea that everything downstream—bits, numbers, arrays—is data wrapped in successive layers of agreed-upon meaning.

Chapter 2

The Philosophy of Representation

What it means for one thing to stand for another; concrete versus abstract representation and the historical shift from tally marks to abstract number systems; Plato's allegory of the cave and the idea that representations can be incomplete or misleading; Aristotle's more practical view of mental representation as a useful internal model rather than a perfect copy; medieval and modern philosophical treatments of representation (Aquinas, Descartes, Kant, Sartre) and their relevance to encoding and interpretation; the layered nature of representation, where symbols represent symbols all the way down to physical reality; the abstraction hierarchy in computing, from physical voltages to applications, and where arrays sit within it; Shannon's information theory as the mathematical formalization of representation, encoding, decoding, and the bit as the fundamental unit of distinguishing information.

Chapter 3

Voltage to Bits

How transistors encode binary states, voltage levels, noise margins, signal integrity, why binary won over ternary.

Chapter 4

Binary Representation and Boolean Algebra

Two-state logic, Boolean operations, truth tables, De Morgan's laws, bit as fundamental unit.

Chapter 5

Number Systems and Bases

Positional notation, decimal, binary, octal, hexadecimal. Base conversion algorithms. Historical development. Why different bases matter for different purposes.

Chapter 6

Integer Representation: Unsigned

Binary positional notation, range limitations, conversion algorithms, hexadecimal convenience.

Chapter 7

Signed Integer Representation

Sign-magnitude, one's complement, two's complement (why it won), arithmetic operations, overflow detection.

Chapter 8

Integer Overflow and Wraparound Behavior

Undefined behavior, modular arithmetic, detecting overflow, saturating arithmetic, language-specific behaviors.

Chapter 9

Fixed-Point Representation

Q-format notation, scaling factors, precision-range tradeoffs, embedded systems applications, fixed-point arithmetic.

Chapter 10

Floating-Point Representation: IEEE 754

Sign, exponent, mantissa encoding, normalized and denormalized numbers, single vs. double precision.

Chapter 11

Special Floating-Point Values

Infinity (positive and negative), NaN (quiet and signaling), signed zero, subnormal numbers.

Chapter 12

Floating-Point Arithmetic Operations

Addition, multiplication, division, rounding modes (round to nearest, toward zero, toward), fused multiply-add.

Chapter 13

Floating-Point Error Analysis

Precision loss, catastrophic cancellation, machine epsilon, relative error, Kahan summation algorithm.

Chapter 14

Decimal Floating-Point Representation

IEEE 754-2008 decimal formats, financial computing requirements, densely packed decimal.

Chapter 15

Extended Precision and Arbitrary Precision

80-bit extended precision, quadruple precision (128-bit), arbitrary precision libraries (GMP, MPFR).

Chapter 16

Character Encoding: ASCII and Extensions

7-bit ASCII, extended ASCII variants, code pages, limitations for internationalization.

Chapter 17

Unicode: Universal Character Encoding

Code points, planes, combining characters, grapheme clusters, normalization forms (NFC, NFD, NFKC, NFKD).

Chapter 18

Unicode Transformation Formats

UTF-8 (variable length, backward compatible), UTF-16 (surrogate pairs), UTF-32 (fixed length), BOM issues.

Chapter 19

Text Processing Complexities

Grapheme vs. code point counting, case folding, locale-dependent operations, text segmentation.

Chapter 20

Endianness: Byte Ordering

Big-endian vs. little-endian, historical reasons, network byte order, bi-endian systems, byte swapping.

Chapter 21

Cross-Platform Data Exchange

Serialization formats, portable binary formats, protocol design considerations.

Chapter 22

Bitwise Operations Fundamentals

AND, OR, XOR, NOT operations, truth tables, bit manipulation primitives.

Chapter 23

Bit Shifting and Rotation

Logical shift, arithmetic shift, rotate left/right, applications to multiplication/division by powers of 2.

Chapter 24

Bit Manipulation Techniques

Setting/clearing/toggling bits, bit masks, extracting bit fields, counting set bits (population count).

Chapter 25

Bit Packing and Flags

Packing multiple boolean flags, bitfields in structs, union tricks, bit arrays.

Chapter 26

Advanced Bit Hacking

Finding rightmost set bit, isolating bit patterns, bit reversal, Gray code conversion.

Chapter 27

Data Alignment Fundamentals

Natural alignment, alignment requirements by type, misalignment penalties.

Chapter 28

Structure Padding and Layout

Compiler padding rules, struct member ordering, packing pragmas, cache line awareness.

Chapter 29

Alignment Optimization Techniques

Manual padding, alignment attributes, reordering for cache efficiency.

Chapter 30

Color Representation Models

RGB, RGBA, HSV, HSL, CMYK, YUV, color spaces, gamma correction.

Chapter 31

Pixel Formats and Bit Depth

1-bit, 8-bit indexed, 16-bit (RGB565), 24-bit, 32-bit (with alpha), HDR formats.

Chapter 32

Image Compression Fundamentals

Lossless (PNG, GIF) vs. lossy (JPEG), compression ratios, quality tradeoffs.

Chapter 33

Video Encoding Principles

Frame types (I, P, B), motion compensation, codecs (H.264, H.265, VP9, AV1).

Chapter 34

Audio Representation: Sampling Theory

Nyquist theorem, sample rate, bit depth, quantization noise, aliasing.

Chapter 35

Audio Encoding Formats

PCM (uncompressed), FLAC (lossless), MP3, AAC, Opus (lossy), perceptual coding.

Chapter 36

Signal Processing Representations

Time domain, frequency domain, spectrograms, wavelet transforms.

Chapter 37

Pointer Representation

How pointers are stored in memory, pointer size (32-bit vs. 64-bit), pointer tagging.

Chapter 38

Pointer Arithmetic and Address Calculation

Array element addressing, struct member offsets, pointer subtraction.

Chapter 39

Special Pointer Values

Null pointers, wild pointers, dangling pointers, pointer alignment requirements.

Chapter 40

Error Detection Codes

Parity bits (even/odd), checksums, CRC algorithms, hash-based integrity.

Chapter 41

Error Correction Codes

Hamming codes, Reed-Solomon codes, LDPC codes, convolutional codes.

Chapter 42

Cyclic Redundancy Checks (CRC)

CRC polynomials, CRC-8, CRC-16, CRC-32 standards, efficient implementation using tables, applications in data integrity.

Chapter 43

Data Serialization Fundamentals

Converting in-memory structures to byte streams, portability issues.

Chapter 44

Binary Serialization Formats

Protocol Buffers, FlatBuffers, Cap'n Proto, MessagePack, CBOR.

Chapter 45

Text Serialization Formats

JSON, XML, YAML, TOML, human-readable vs. machine-efficient tradeoffs.

Chapter 46

Custom Binary Protocols

Designing efficient binary formats, alignment considerations, versioning.

Chapter 47

Data Compression Theory

Information theory basics, entropy, lossless vs. lossy bounds.

Chapter 48

Dictionary-Based Compression

LZ77, LZ78, LZW, Lempel-Ziv family, dictionary construction.

Chapter 49

Entropy Coding

Huffman coding, arithmetic coding, asymmetric numeral systems (ANS).

Chapter 50

Modern Compression Algorithms

zlib, gzip, bzip2, LZMA, Zstandard, Brotli, compression levels.

Chapter 51

Specialized Compression

Run-length encoding, delta encoding, columnar compression, domain-specific methods.

Part II

Architecture & Circuitry

ARRAYS EXIST *in physical hardware. To understand array performance, we must understand the machines that execute our code—from individual transistors switching in response to a voltage, up through logic gates, memory cells, processors, caches, and the massively parallel engines—multicore CPUs, vector units, and GPUs—that move and transform arrays today. This volume builds that complete picture, one layer at a time, continuing directly from where Volume I left off.*

What Makes This Different:

- ***Silicon to Software:*** *Complete vertical integration, continuing directly from Volume I's voltage-to-bits foundation*
- ***Modern Architecture:*** *Multi-core, SIMD, GPU, and heterogeneous systems treated as natural extensions of single-unit hardware*
- ***Performance Foundations:*** *Why code runs fast or slow, at every scale*
- ***Hardware-Software Contract:*** *What each layer assumes, and what arrays can rely on in Volume III*

> *"The purpose of computing is insight, not numbers."*
>
> — RICHARD HAMMING

Chapter 52

Semiconductor Physics Foundations

Silicon properties, doping (n-type, p-type), P-N junctions, depletion regions, how semiconductors enable switching.

Chapter 53

MOSFET Transistors

Gate, source, drain, channel, threshold voltage, NMOS and PMOS, how transistors act as switches.

Chapter 54

CMOS Logic Fundamentals

Complementary MOS, pull-up and pull-down networks, static power vs. dynamic power, why CMOS won.

Chapter 55

Logic Gate Implementation

CMOS implementation of NOT, NAND, NOR gates, gate delay, rise/fall time, fan-out limitations.

Chapter 56

Logic Gate Families

AND, OR, XOR, XNOR from NAND/NOR, transmission gates, tri-state buffers.

Chapter 57

Combinational Circuit Design

Truth tables to logic expressions, Karnaugh maps, minimization, multi-level logic.

Chapter 58

Arithmetic Circuits: Adders

Half adder, full adder, ripple-carry adder, carry-lookahead adder, carry-select adder.

Chapter 59

Arithmetic Circuits: Multipliers and Dividers

Array multiplier, Booth multiplier, Wallace tree, division algorithms, restoring vs. non-restoring.

Chapter 60

Multiplexers and Demultiplexers

2:1 mux, 4:1 mux, tree structures, using muxes for logic implementation.

Chapter 61

Encoders and Decoders

Binary encoding, priority encoding, 7-segment decoder, address decoding.

Chapter 62

Comparators and Magnitude Detection

Equality comparator, magnitude comparator, signed vs. unsigned comparison.

Chapter 63

Advanced Arithmetic Units

Multipliers (array multiplier, Booth's algorithm, Wallace tree), dividers, floating-point units (FPUs), fused multiply-add (FMA).

Chapter 64

Sequential Logic Fundamentals

Difference between combinational and sequential logic, feedback, state, clock signals, synchronous versus asynchronous design.

Chapter 65

Latches: SR, D Latches

Set-reset latch, gated D latch, transparent latch, latch vs. flip-flop distinction.

Chapter 66

Flip-Flops: D, JK, T Flip-Flops

Edge-triggered flip-flops, master-slave configuration, setup and hold time, clock-to-Q delay.

Chapter 67

Registers and Register Files

Parallel-load register, shift registers (SISO, SIPO, PISO, PIPO), register banks for CPU.

Chapter 68

Counters and Timers

Ripple counter, synchronous counter, up/down counter, modulo-N counter, timer circuits.

Chapter 69

Finite State Machines

Moore vs. Mealy machines, state diagrams, state table, state minimization, FSM implementation.

Chapter 70

Memory Cell Design: SRAM

6-transistor SRAM cell, read and write operations, stability analysis, SRAM array organization.

Chapter 71

Memory Cell Design: DRAM

1-transistor 1-capacitor cell, charge storage, sense amplifiers, refresh requirement.

Chapter 72

Memory Array Organization

Row and column decoding, wordline and bitline, memory cell array structure, address multiplexing.

Chapter 73

DRAM Operation and Timing

Row activation, column access, precharge, timing parameters (tRCD, tRP, tRAS, CAS latency).

Chapter 74

DRAM Generations

SDR SDRAM, DDR, DDR2, DDR3, DDR4, DDR5, bandwidth evolution, power efficiency.

Chapter 75

Advanced DRAM Features

Burst mode, bank interleaving, dual-channel, command queuing, on-die termination.

Chapter 76

Non-Volatile Memory Technologies

Flash memory (NAND, NOR), program/erase cycles, wear leveling, EEPROM, ROM variants.

Chapter 77

Flash Memory Operations

Programming (writing), erasing, read operations, wear leveling, write amplification, garbage collection, over-provisioning.

Chapter 78

Emerging Memory Technologies

Phase-change memory (PCM), resistive RAM (ReRAM), magnetoresistive RAM (MRAM), 3D XPoint.

Chapter 79

Memory Hierarchy Fundamentals

Pyramid structure, latency-capacity tradeoffs, why hierarchies exist, locality principles.

Chapter 80

Cache Memory Fundamentals

Temporal locality, spatial locality, cache line (block), tag-data organization.

Chapter 81

Cache Mapping Strategies

Direct-mapped cache, fully-associative cache, set-associative cache (2-way, 4-way, 8-way, N-way).

Chapter 82

Cache Replacement Policies

LRU (least recently used), LFU, FIFO, random, pseudo-LRU, practical implementations.

Chapter 83

Cache Write Policies

Write-through, write-back, write-allocate, no-write-allocate, dirty bits.

Chapter 84

Multi-Level Cache Hierarchies

L1 instruction and data caches, unified L2 cache, L3 shared cache, inclusive vs. exclusive caches.

Chapter 85

Victim Caches and Advanced Structures

Victim cache for conflict misses, stream buffers for prefetching, trace caches.

Chapter 86

Cache Coherence Problem

Shared memory multiprocessors, cache inconsistency, coherence vs. consistency.

Chapter 87

Cache Coherence Protocols: MESI

Modified, Exclusive, Shared, Invalid states, state transitions, bus snooping.

Chapter 88

Cache Coherence Protocols: MOESI and Beyond

Owned state, directory-based coherence, scalability issues.

Chapter 89

Cache Performance Analysis

Hit rate, miss rate, average memory access time (AMAT), miss penalty, classifying misses (compulsory, capacity, conflict).

Chapter 90

Cache Optimization Techniques

Blocking/tiling, array padding, loop interchange, prefetching, cache-aware algorithms.

Chapter 91

NUMA Architecture

Non-uniform memory access, local vs. remote memory, latency differences, NUMA domains.

Chapter 92

NUMA-Aware Programming

First-touch policy, memory binding (numactl), thread affinity, optimizing placement.

Chapter 93

Instruction Set Architecture (ISA)

RISC versus CISC philosophy, instruction encoding and formats, addressing modes, register architecture, condition codes.

Chapter 94

x86 Architecture

x86 instruction encoding, variable-length instructions, complex addressing modes, backward compatibility, x86-64 extensions.

Chapter 95

ARM Architecture

Load-store architecture, fixed-length instructions (ARM mode), Thumb instruction set, NEON SIMD, ARM versus x86 comparison.

Chapter 96

RISC-V Architecture

Open ISA, modular extensions, base integer instruction set, standard extensions (M, A, F, D, C), design philosophy.

Chapter 97

Register Architecture

General-purpose registers, special-purpose registers (PC, SP, flags), register windows, register renaming.

Chapter 98

Addressing Modes

Immediate, register direct, memory direct, register indirect, indexed, PC-relative, stack addressing.

Chapter 99

Instruction Encoding

Fixed-length vs. variable-length, opcode, operands, prefix bytes (x86), instruction alignment.

Chapter 100

Pipelining Fundamentals

Instruction pipeline stages (IF, ID, EX, MEM, WB), throughput improvement, latency, ideal CPI, pipeline diagrams.

Chapter 101

Pipeline Hazards

Structural hazards, data hazards (RAW, WAR, WAW), control hazards, stalls and bubbles, hazard detection and resolution.

Chapter 102

Data Forwarding and Bypassing

Forwarding paths, bypassing results from later stages, forwarding logic, load-use hazard, limitations of forwarding.

Chapter 103

Pipeline Stalls and Bubbles

When forwarding isn't enough, NOP insertion, performance impact.

Chapter 104

Branch Handling in Pipelines

Branch penalty, delayed branches, branch prediction necessity, flushing pipeline on misprediction, branch delay slots.

Chapter 105

Branch Prediction Fundamentals

Static prediction, dynamic prediction, branch history, importance for performance.

Chapter 106

Branch Prediction: Simple Schemes

1-bit predictor, 2-bit saturating counter, bimodal predictor.

Chapter 107

Branch Prediction: Advanced Schemes

Two-level adaptive predictors, local vs. global history, correlating predictors.

Chapter 108

Branch Target Buffer and Return Stack

BTB for target prediction, return address stack (RAS) for function returns, indirect branch prediction.

Chapter 109

Speculative Execution

Branch speculation, memory speculation, exceptions in speculative execution, precise interrupts, ROB and commit.

Chapter 110

Speculative Execution Vulnerabilities

Spectre attacks (branch target injection, bounds check bypass), Meltdown (rogue data cache load), side-channel exploitation, mitigations.

Chapter 111

Out-of-Order Execution

Tomasulo's algorithm, reservation stations, register renaming, reorder buffer (ROB), commit stage, speculative execution.

Chapter 112

Register Renaming

Eliminating false dependencies (WAR, WAW), physical versus architectural registers, register alias table (RAT), freelist management.

Chapter 113

Instruction Scheduling

Issue width, instruction window, dependency checking, wake-up and select logic.

Chapter 114

Superscalar Execution

Multiple issue, dispatch width, execution units, commit width, IPC (instructions per cycle).

Chapter 115

Very Long Instruction Word (VLIW)

Static scheduling by compiler, explicit parallelism, no dependency checking hardware, Itanium example.

Chapter 116

Simultaneous Multithreading (SMT)

Hyper-threading, thread-level parallelism, sharing execution resources, Intel and AMD implementations.

Chapter 117

Memory Ordering and Consistency

Load/store reordering, memory fences, barriers, acquire-release semantics.

Chapter 118

Consistency Models

Sequential consistency, total store order (TSO), weak ordering, relaxed models.

Chapter 119

Virtual Memory Fundamentals

Virtual address space, physical address space, address translation, memory protection.

Chapter 120

Page Tables and Address Translation

Page table entry structure, multi-level page tables, page table walk.

Chapter 121

Translation Lookahead Buffer (TLB)

TLB structure, TLB hit/miss, TLB reach, large pages (huge pages, superpages).

Chapter 122

Page Replacement Algorithms

Optimal (Belady), FIFO, LRU, clock (second chance), working set model.

Chapter 123

Demand Paging and Page Faults

Page fault handling, major vs. minor faults, swap space, thrashing.

Chapter 124

Memory Management Unit (MMU)

Hardware implementation, page table base register, protection bits, privilege levels.

Chapter 125

Memory Protection and Isolation

Process isolation, kernel vs. user space, protection rings, segmentation.

Chapter 126

Input/Output Systems

I/O devices and controllers, I/O address space (port-mapped versus memory-mapped I/O), programmed I/O, interrupt-driven I/O.

Chapter 127

Interrupt Handling

Interrupt request (IRQ), interrupt vector table, interrupt service routine (ISR), interrupt priority, nested interrupts, interrupt latency.

Chapter 128

Exception Handling

Faults, traps, aborts, exception vectors, privilege level transitions.

Chapter 129

Direct Memory Access (DMA)

DMA controller, bus mastering, DMA transfer modes (burst, cycle stealing), scatter-gather DMA, reducing CPU overhead.

Chapter 130

I/O Memory Management Unit (IOMMU)

Device address translation, DMA protection, virtualization support, IOMMU page tables.

Chapter 131

Bus Architecture and Protocols

System bus, memory bus, I/O bus, bus arbitration, split transactions.

Chapter 132

Interconnect Fabrics

Point-to-point links, crossbar switches, mesh networks, ring interconnects.

Chapter 133

PCIe Architecture

Lanes, link width, generations (1.0-6.0), packets and transactions, TLP/DLLP/Physical layers.

Chapter 134

Cache-Coherent Interconnects

Intel QPI/UPI, AMD Infinity Fabric, coherence traffic, non-uniform cache access (NUCA).

Chapter 135

Network-on-Chip (NoC)

On-chip routing, wormhole routing, virtual channels, deadlock avoidance.

Chapter 136

Storage Controllers and Interfaces

SATA, SAS, NVMe, NVMe over Fabrics, command queuing, storage protocol stack, queue depth and parallelism.

Chapter 137

Multi-Core Processors

Chip multiprocessors (CMP), symmetric multiprocessing (SMP), shared L2/L3 caches, on-chip interconnect (ring, mesh, crossbar).

Chapter 138

SIMD and Vector Processing

Single instruction multiple data, vector registers, lane-based execution, packed operations.

Chapter 139

x86 SIMD Extensions

MMX, SSE, SSE2-SSE4, AVX, AVX2, AVX-512, register width evolution.

Chapter 140

SIMD Programming: Intrinsics

Compiler intrinsics, vector types, operations, portability challenges.

Chapter 141

SIMD Programming: SSE/AVX

x86 SIMD instruction sets, 128-bit (SSE), 256-bit (AVX), 512-bit (AVX-512).

Chapter 142

ARM NEON and SVE

NEON instruction set, Scalable Vector Extension, predication, variable vector length.

Chapter 143

SIMD Programming: ARM NEON

ARM SIMD, 128-bit vectors, intrinsics, mobile/embedded optimization.

Chapter 144

SIMD: Alignment Requirements

Aligned loads/stores, performance penalties for misalignment, ensuring alignment.

Chapter 145

SIMD: Masked Operations

Conditional execution, AVX-512 mask registers, handling irregular arrays.

Chapter 146

Autovectorization

Compiler automatic vectorization, loop dependencies, enabling vectorization.

Chapter 147

Vectorization: Loop Dependencies

Data dependencies preventing vectorization, loop-carried dependencies, array privatization.

Chapter 148

Vectorization: Compiler Hints

Pragmas (ivdep, vector), assume aligned, restrict keyword, helping the compiler.

Chapter 149

GPU Architecture Overview

Massively parallel architecture, streaming multiprocessors, SIMT execution model.

Chapter 150

GPU Architecture Deep Dive

Streaming multiprocessors (SMs), CUDA cores, warp schedulers, memory hierarchy.

Chapter 151

GPU Execution Model

Threads, warps/wavefronts, thread blocks, grid, kernel launch, occupancy.

Chapter 152

GPU Execution Model: CUDA

Kernel launches, thread hierarchy (thread, warp, block, grid), device functions.

Chapter 153

GPU Thread Organization

blockIdx, threadIdx, blockDim, gridDim, mapping threads to data.

Chapter 154

GPU Memory Hierarchy

Global memory, shared memory, registers, constant memory, texture memory, L1/L2 caches.

Chapter 155

GPU Global Memory Access Patterns

Coalesced access, strided access, memory transactions, alignment requirements.

Chapter 156

GPU Shared Memory

On-chip memory, fast access, bank conflicts, tiling for shared memory.

Chapter 157

GPU Bank Conflicts

Shared memory banks, conflict-free access patterns, broadcast, padding to avoid conflicts.

Chapter 158

GPU Synchronization

`__syncthreads()`, block-level synchronization, warp-level primitives, atomics.

Chapter 159

GPU Atomic Operations

atomicAdd, atomicCAS, global vs. shared memory atomics, performance implications.

Chapter 160

GPU Occupancy

Active warps per SM, register usage, shared memory usage, occupancy calculator.

Chapter 161

GPU Register Pressure

Register spilling, local memory, reducing register usage, occupancy tradeoff.

Chapter 162

GPU Warp Divergence

Branch divergence, serialized execution, masking, minimizing divergence.

Chapter 163

GPU Kernel Optimization

Maximizing occupancy, minimizing divergence, memory coalescing, arithmetic intensity.

Chapter 164

GPU Streams and Concurrency

Asynchronous execution, overlapping compute and memory transfer, stream priorities.

Chapter 165

GPU Multi-Stream Processing

Concurrent kernel execution, data dependencies, stream synchronization.

Chapter 166

GPU Memory Transfer Optimization

Pinned memory, async transfers, batching, minimizing host-device communication.

Chapter 167

GPU Unified Memory

Automatic data migration, page faults, prefetching, simplifying memory management.

Chapter 168

GPU Programming Models

CUDA, OpenCL, compute shaders, host-device interaction, kernel code.

Chapter 169

GPU Profiling Tools

nvprof, Nsight Compute, Nsight Systems, identifying bottlenecks.

Chapter 170

TPU Architecture

Systolic array for matrix multiplication, matrix unit, on-chip memory, dataflow.

Chapter 171

Power Management Mechanisms

Dynamic voltage and frequency scaling (DVFS), clock gating, power gating, C-states, P-states.

Chapter 172

Thermal Design and Throttling

Thermal design power (TDP), temperature monitoring, thermal throttling, cooling solutions.

Chapter 173

Hardware Performance Counters

Performance monitoring units (PMU), counting events (cycles, instructions, cache misses, branch mispredictions).

Chapter 174

Profiling with Hardware Counters

Perf tools, VTune, hardware event sampling, attributing performance to code.

Chapter 175

Hardware Security Features

Secure boot, trusted execution environments (SGX, TrustZone), memory encryption (SEV, TME).

Chapter 176

Spectre and Meltdown

Speculative execution vulnerabilities, side-channel attacks, mitigations (KPTI, retpoline, microcode updates).

Chapter 177

Memory Safety Mechanisms

Bounds checking (Intel MPX), pointer authentication (ARM PAC), tagged memory, stack canaries.

Chapter 178

Design for Testability

Scan chains, built-in self-test (BIST), fault injection, manufacturing test.

Chapter 179

Hardware Description Languages

Verilog and SystemVerilog, VHDL, behavioral vs. structural modeling, simulation vs. synthesis.

Chapter 180

Digital Design Flow

RTL design, logic synthesis, place and route, timing closure, physical design.

Chapter 181

FPGA Architecture

Look-up tables (LUTs), configurable logic blocks (CLBs), programmable interconnect, block RAM, DSP slices.

Chapter 182

ASIC vs. FPGA Tradeoffs

Performance, power, cost, flexibility, development time, reconfigurability.

Chapter 183

ASIC Design for Array Operations

Custom chips, matrix multiplication accelerators, Google TPU, specialized silicon.

Glossary

Reflections at the End

As you turn the final pages of *Arliz*, I invite you to pause—just for a moment—and look back. Think about the path you've taken through these chapters. Let yourself ask:

> "Wait… what just happened? What did I actually learn?"

I won't pretend to answer that for you. The truth is—**only you can**. Maybe it was a lot. Maybe it wasn't what you expected. But if you're here, reading this, something must have kept you going. That means something.
This book didn't start with a grand plan. It started with a simple itch: **What even is an array, really?** What began as a curiosity about a "data structure" became something much stranger and—hopefully—much richer. We wandered through history, philosophy, mathematics, logic gates, and machine internals. We stared at ancient stones and modern memory layouts and tried to see the invisible threads connecting them.
If that sounds like a weird journey, well—yeah. It was.

This is Not the End

Arliz isn't a closed book. It's a snapshot. A frame in motion. And maybe your understanding is the same. You'll return to these ideas later, years from now, and see new angles. You'll say, "Oh. That's what it meant." That's good. That's growth.
Everything you've read here is connected to something bigger—algorithms, networks, languages, systems, even the people who built them. There's no finish line. And that's beautiful.

From Me to You

If this book gave you something—an idea, a shift in thinking, a pause to wonder—then it has done its job. If it made you feel like maybe programming isn't just code and rules, but a window into something deeper—then that means everything to me.

Thank you for being here.
Thank you for not skipping the hard parts.
Thank you for choosing to think.

One More Thing

You're not alone in this.
The Arliz project lives on GitHub, and the conversations around it will (hopefully) continue. If you spot mistakes, have better explanations, or just want to say hi—come by. Teach me something. Teach someone else. That's the best way to say thanks.
Knowledge grows in community.
So share. Build. Break. Rebuild.
Ask better questions.
And always, always—stay curious.

Final Words

Arrays were just the excuse.
Thinking was the goal.
And if you've started to think more clearly, more deeply, or more historically about what you're doing when you write code—then this wasn't just a technical book.
It was a human one.
Welcome to the quiet, lifelong joy of understanding.

This completes the first living edition of Arliz.

Thank you for joining this journey from zero to arrays, from ancient counting to modern computation.
The exploration continues...

Author's Notes and Reflections

On Naming Conventions and Creative Processes

I should confess something about my naming process: I tend to pick names first and find meaningful justifications later. Very scientific, I know! The name "Arliz" started as a random choice that simply felt right phonetically. Only after committing to it did I discover the backronym that now defines its meaning. This probably says something about my creative process—intuition first, rationalization second.

This approach extends beyond naming. Many aspects of this book emerged organically from curiosity rather than systematic planning. What began as personal notes to understand arrays evolved into a comprehensive exploration of computational thinking itself.

On Perfectionism and Living Documents

You should know that many of the algorithms presented in this book are my own implementations, built from first principles rather than copied from optimized sources. This means you might encounter code that runs slower than industry standards—or occasionally faster, when serendipity strikes.

Some might view this as a weakness, but I consider it a feature. The goal isn't to provide the most optimized implementations but to demonstrate the thinking process that leads to understanding. When you can reconstruct a solution from fundamental principles, you've achieved something more valuable than memorizing an optimal algorithm.

On Academic Formality and Personal Voice

You might notice that this book alternates between formal academic language and more conversational tones. This is intentional. While I respect the precision that formal writing provides, I also believe that learning happens best in an atmosphere of intellectual friendship rather than academic intimidation.

When I suggest you could "use this book as a makeshift heating device" if you find the approach ridiculous, I'm not being flippant—I'm acknowledging that not every approach works for every learner. Intellectual honesty includes admitting when your methods might not suit your audience.

On Scope and Ambition

The scope of this book—from ancient counting to modern distributed systems—might seem overly ambitious. Some might argue that such breadth necessarily sacrifices depth. I disagree, but I understand the concern.

My experience suggests that understanding connections between disparate fields often provides insights that narrow specialization misses. When you see arrays as part of humanity's broader intellectual project, you understand them differently than when you see them as isolated programming constructs.

That said, if you find the historical sections tedious or irrelevant, you have my permission to skip ahead. The book is designed to be valuable even when read non-sequentially.

On Errors and Imperfection

I mentioned that you'll find errors in this book. This isn't false modesty—it's realistic acknowledgment. Complex explanations, mathematical derivations, and code implementations inevitably contain mistakes, especially in a work that grows and evolves over time.

Rather than viewing this as a flaw, I encourage you to see it as an opportunity for engagement. When you find an error, you're not just identifying a problem—you're participating in the process of building better understanding. The best learning often

happens when we encounter and resolve contradictions.

On Time Investment and Expectations

When I suggest this book requires months rather than weekends to master, I'm not trying to inflate its importance. Complex concepts genuinely require time to internalize. Mathematical intuition develops gradually, through repeated exposure and active practice.

If you're looking for quick solutions to immediate programming problems, this book will frustrate you. If you're interested in developing the kind of deep understanding that serves you throughout your career, the time investment will prove worthwhile.

On Community and Collaboration

This book exists because of community—the open-source community that provides tools and resources, the academic community that develops and refines concepts, and the programming community that applies these ideas in practice.

Your engagement with this material makes you part of that community. Whether you find errors, suggest improvements, or simply work through the exercises thoughtfully, you're contributing to the collective understanding that makes books like this possible.

Final Reflection

Writing this book has been an exercise in understanding my own learning process. I've discovered that I learn best by building connections between disparate ideas, by understanding historical context, and by implementing concepts from scratch rather than accepting them as given.

Your learning process might be entirely different. Use what serves you from this book, adapt what needs adaptation, and don't hesitate to supplement with other resources when my explanations fall short.

The goal isn't for you to learn exactly as I did, but for you to develop your own path to deep understanding.

These notes reflect thoughts and observations that didn't fit elsewhere but seemed worth preserving. They represent the informal side of a formal exploration—the human element in what might otherwise seem like purely technical content.